System & attack model

1.Overview

This model represents the complete lifecycle of RC4 encryption, including normal operation, bias exploitation by an attacker, and prevention using the Drop-N technique.

The system first performs RC4 encryption using the Key Scheduling Algorithm and Pseudo Random Generation Algorithm. The attacker observes ciphertext samples and performs frequency analysis to exploit initial keystream bias. Finally, the Drop-N prevention method reduces bias and prevents successful attacks.

Phase 1 — Normal RC4 Operation

The RC4 system receives plaintext input from the user. A secret key is generated and used by the Key Scheduling Algorithm (KSA) to initialize the permutation array S.

The Pseudo Random Generation Algorithm (PRGA) then generates keystream bytes. These bytes are XORed with plaintext to produce ciphertext.

However, the initial keystream bytes contain statistical bias.

Phase 2 — Attack Phase

The attacker passively observes multiple ciphertext messages generated using RC4.

Since RC4 initial keystream bytes are biased, some byte values occur more frequently than expected.

The attacker performs frequency analysis and identifies the most probable keystream byte.

Using this predicted value:

$$Plaintext = Ciphertext \oplus Keystream$$

the attacker recovers plaintext characters.

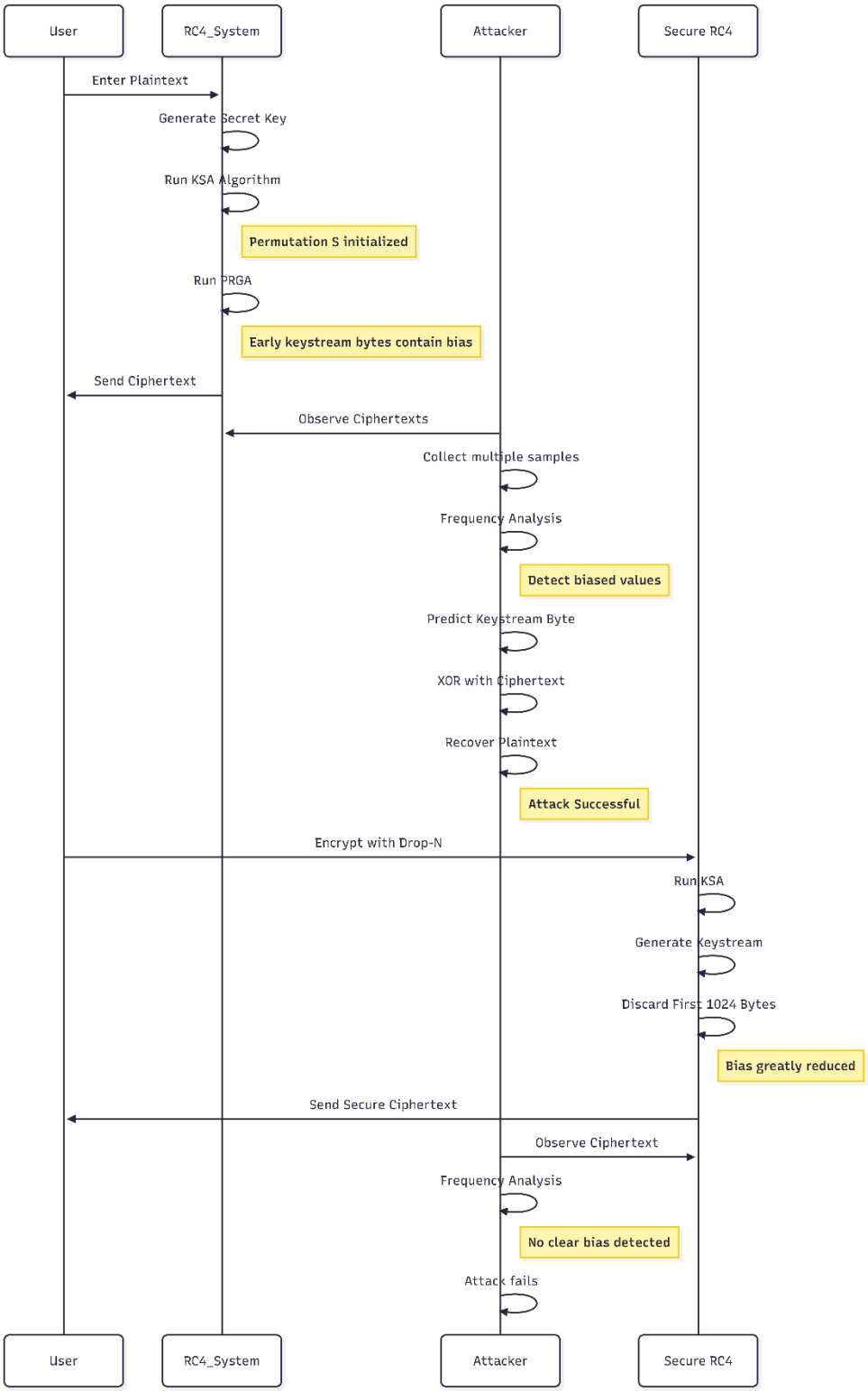
Phase 3 — Prevention Phase

To prevent the attack, the Drop-N technique is applied.

The RC4 system generates keystream bytes normally but discards the first 1024 bytes before encryption.

After dropping the initial bytes, the permutation array becomes more randomized and keystream bias is reduced.

The attacker performs frequency analysis again but cannot detect useful patterns, and the attack fails.


User
RC4_System
Attacker
Secure RC4
Enter Plaintext
Generate Secret Key
Run KSA Algorithm
Permutation S initialized
Run PRGA
Early keystream bytes contain bias
Send Ciphertext
Observe Ciphertexts
Collect multiple samples
Frequency Analysis
Detect biased values
Predict Keystream Byte
XOR with Ciphertext
Recover Plaintext
Attack Successful
Encrypt with Drop-N
Run KSA
Generate Keystream
Discard First 1024 Bytes
Bias greatly reduced
Send Secure Ciphertext
Observe Ciphertext
Frequency Analysis
No clear bias detected
Attack fails
User
RC4_System
Attacker
Secure RC4

2.code flow

Explanation

The Python codebase simulates an **RC4 Initial Keystream Bias Attack**. The system side (rc4_system.py) implements RC4 from scratch using the **Key Scheduling Algorithm (KSA)** and **Pseudo Random Generation Algorithm (PRGA)** to generate keystream bytes.

During execution, the system generates multiple RC4 keystream samples using randomly generated secret keys. For each encryption session, the **first keystream byte is recorded**, because RC4 bias is strongest at the beginning of the keystream.

These first-byte values are saved into **samples.txt**, representing the data that an attacker could observe from multiple encrypted messages.

Attacker Phase

The attacker program (rc4_attacker.py) represents the attacker's perspective.

It reads the observed keystream samples from **samples.txt** and performs **frequency analysis** to determine which byte appears most often.

Because RC4 has statistical bias, one byte tends to appear more frequently than expected. The attacker assumes this **most frequent byte is the likely keystream value**.

Using this predicted keystream byte, the attacker attempts to recover the plaintext message.

3.Pseudo-Algorithm for Attack

Numbered Steps

Algorithm: RC4 Initial Keystream Bias Attack

Inputs:
Multiple encrypted sessions generated using RC4 with different secret keys

Outputs:
Predicted keystream byte and partially recovered plaintext

Step 1 — Generate RC4 Keystream Samples

The system generates multiple RC4 encryption sessions using randomly selected secret keys. For each session, the Key Scheduling Algorithm initializes the permutation array and the Pseudo Random Generation Algorithm produces keystream bytes. The first keystream byte from each session is recorded and stored as a sample for analysis.

Step 2 — Collect Statistical Data

The attacker reads the stored keystream samples and counts how many times each byte value appears. Since RC4 has known statistical biases in early keystream bytes, some byte values occur more frequently than expected.

Step 3 — Identify Most Frequent Byte

The attacker determines the byte value with the highest frequency among all samples. This byte is considered the most likely value of the first keystream byte because RC4 bias causes certain values to appear more often.

Step 4 — Predict Keystream Value

The attacker assumes that the most frequent byte represents the probable keystream byte used during encryption. This predicted value is then used for message recovery.

Step 5 — Encrypt Demonstration Message

A plaintext message is converted into ASCII values and encrypted using the XOR operation with the predicted keystream byte to produce ciphertext values.

Step 6 — Recover Plaintext

The attacker applies the XOR operation between the ciphertext and the predicted keystream byte. Because XOR is reversible, the original plaintext characters are recovered.

Step 7 — Demonstrate Confidentiality Weakness

If the recovered plaintext matches the original message, the attacker demonstrates that RC4 keystream bias allows partial recovery of encrypted data without knowledge of the secret key.